Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:

Студент 4 курса

Группы ИИ-26

Петручик Д. А.

Проверил:

Никитин С. К.

Брест 2026

Цель работы: исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Ход работы:

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:

- **Конфигурация A (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
- **Конфигурация B (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).

4. Критерий остановки для обеих конфигураций: суммарная ошибка $Es \le Ee$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
   - Количество эпох до критерия остановки.
   - Итоговую суммарную ошибку и accuracy.
   - Среднюю абсолютную ошибку в исходной шкале [c0; c1].
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
   - График сходимости (суммарная ошибка Es от эпохи) для обеих конфигураций на одних осях.
   - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации A и B.
8. Визуализация разделяющей поверхности:
   - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
   - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).
9. Реализовать режим функционирования сети:
   - Пользователь вводит (A, B) из [−10; 10].
   - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале [c0; c1], класс.
   - Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.
10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).
11. Вывод: ответить на все вопросы и провести анализ результатов.

Вариант 10 (c0 = −3, c1 = −8)

A	B	A ⊕ B
−3	−3	−3
−3	−8	−8
−8	−3	−8
−8	−8	−3

Код программы:

```
import sys
import numpy as np
import matplotlib.pyplot as plt

c0, c1 = -3.0, -8.0

X = np.array([
    [-3, -3],
    [-3, -8],
    [-8, -3],
    [-8, -8]
], dtype=float)

Y_raw = np.array([
    [-3],
    [-8],
    [-8],
    [-3]
], dtype=float)

def normalize(x, lo, hi):
    return (x - lo) / (hi - lo)

def denormalize(y, lo, hi):
    return lo + y * (hi - lo)

Xn = normalize(X, c0, c1)
Yn = normalize(Y_raw, c0, c1)

EPS = 1e-12

def sigmoid(s):
    s = np.clip(s, -500, 500)
    return 1.0 / (1.0 + np.exp(-s))

def dsigmoid(y):
    return y * (1.0 - y)

def relu(s):
    return np.maximum(0.0, s)

def drelu(s):
    return (s > 0).astype(float)
```

```
class MLP221:

    def __init__(self, hidden="sigmoid", lr=0.5, seed=7):
        rng = np.random.RandomState(seed)
        self.W1 = rng.uniform(-0.1, 0.1, (2, 2))
        self.T1 = rng.uniform(-0.1, 0.1, (1, 2))
        self.W2 = rng.uniform(-0.1, 0.1, (2, 1))
        self.T2 = rng.uniform(-0.1, 0.1, (1, 1))

        self.lr = lr
        self.hidden = hidden

        self.rng = np.random.RandomState(seed + 1000)

    def forward(self, x):
        x = np.asarray(x, dtype=float).reshape(1, -1)
        s1 = x @ self.W1 - self.T1

        if self.hidden == "relu":
            y1 = relu(s1)
        else:
            y1 = sigmoid(s1)
        y2 = sigmoid(y1 @ self.W2 - self.T2)

        return s1, y1, y2

    def error(self, y, e):
        y = np.clip(y, EPS, 1.0 - EPS)
        return -np.sum(
            e * np.log(y) +
            (1.0 - e) * np.log(1.0 - y)
        )

    def train_step(self, x, e):
        x = x.reshape(1, -1)
        e = e.reshape(1, -1)

        s1, y1, y2 = self.forward(x)
        delta2 = y2 - e
        back = delta2 @ self.W2.T
        if self.hidden == "relu":
            d1 = drelu(s1)
        else:
            d1 = dsigmoid(y1)

        delta1 = back * d1

        self.W2 -= self.lr * (y1.T @ delta2)
        self.T2 += self.lr * delta2
        self.W1 -= self.lr * (x.T @ delta1)
        self.T1 += self.lr * delta1

        return self.error(y2, e)

    def predict(self, x):
```

```
        return self.forward(x)[2][0, 0]

    def fit(self, X, Y, Ee, max_epochs=20000):
        hist = []

        for epoch in range(1, max_epochs + 1):
            idx = self.rng.permutation(len(X))

            Es = sum(
                self.train_step(X[i], Y[i])
                for i in idx
            )

            hist.append(Es)

            if Es <= Ee:
                return epoch, Es, True, hist

        return max_epochs, Es, False, hist

def accuracy(m, Xn, Y):
    predictions = [
        int(m.predict(x) >= 0.5)
        for x in Xn
    ]

    targets = [
        int(y[0] >= 0.5)
        for y in Y
    ]

    return np.mean(
        np.array(predictions) == np.array(targets)
    )

def mae_orig(m, Xn, Yraw):
    predictions = np.array([
        denormalize(m.predict(x), c0, c1)
        for x in Xn
    ])

    return float(
        np.mean(np.abs(predictions - Yraw.ravel()))
    )

def final_Es(m, Xn, Y):
    return sum(
        m.error(m.forward(x)[2], y.reshape(1, -1))
        for x, y in zip(Xn, Y)
    )

def infer(m, A, B, name):
    xn = normalize(
        np.array([A, B], dtype=float),
        c0,
        c1
```

```
    )

    yn = m.predict(xn)
    yh = denormalize(yn, c0, c1)

    cls = c0 if abs(yh - c0) <= abs(yh - c1) else c1

    print(
        f"[{name}] A={A:g}, B={B:g} -> "
        f"y_norm={yn:.4f}, "
        f"y_hat={yh:.4f}, "
        f"класс {cls:g}"
    )

CONFIGS = {
    "A (Sigmoid)": {
        "hidden": "sigmoid",
        "Ee": 0.05
    },

    "B (ReLU)": {
        "hidden": "relu",
        "Ee": 0.05
    }
}

SEEDS = [1, 2, 3, 4, 5]
MAX_EPOCHS = 20000

colors = {
    "A (Sigmoid)": "tab:blue",
    "B (ReLU)": "tab:red"
}

def run(cfg, seed):
    m = MLP221(
        hidden=cfg["hidden"],
        seed=seed
    )

    ep, Es, ok, hist = m.fit(
        Xn,
        Yn,
        cfg["Ee"],
        MAX_EPOCHS
    )

    result = {
        "seed": seed,
        "epochs": ep,
        "Es": final_Es(m, Xn, Yn),
        "ok": ok,
        "acc": accuracy(m, Xn, Yn),
        "mae": mae_orig(m, Xn, Y_raw),
        "hist": hist
    }
```

```
    return m, result

if __name__ == "__main__":

    res = {}
    models = {}

    for name, cfg in CONFIGS.items():
        res[name] = []
        models[name] = []

        for seed in SEEDS:
            m, r = run(cfg, seed)

            res[name].append(r)
            models[name].append(m)

        print(
            f"\nКонфигурация {name}, Ee={cfg['Ee']}"
        )

        print(
            "seed | эпохи | Es | accuracy | "
            "MAE[c0;c1] | достигнут Ee"
        )

        for r in res[name]:
            print(
                f"{r['seed']:>4} | "
                f"{r['epochs']:>5} | "
                f"{r['Es']:.5f} | "
                f"{r['acc']:.2f} | "
                f"{r['mae']:.4f} | "
                f"{r['ok']}"
            )

    print("\nСводная таблица (5 запусков)")

    print(
        f"{'Конфигурация':<14}|"
        f"{'Сошлось':>8}|"
        f"{'Эпохи min':>10}|"
        f"{'max':>7}|"
        f"{'mean':>8}|"
        f"{'std':>7}|"
        f"{'Es mean':>9}|"
        f"{'acc mean':>9}|"
        f"{'MAE mean':>9}|"
        f"{'MAE std':>8}"
    )

    for name in CONFIGS:
        R = res[name]
        ep = [r["epochs"] for r in R]
```

```
    print(
        f"{name:<14}|"
        f"{sum(r['ok'] for r in R):>6}/5|"
        f"{min(ep):>10}|"
        f"{max(ep):>7}|"
        f"{np.mean(ep):>8.0f}|"
        f"{np.std(ep):>7.0f}|"
        f"{np.mean([r['Es'] for r in R]):>9.4f}|"
        f"{np.mean([r['acc'] for r in R]):>9.2f}|"
        f"{np.mean([r['mae'] for r in R]):>9.4f}|"
        f"{np.std([r['mae'] for r in R]):>8.4f}"
    )

rep = {}

for name in CONFIGS:
    successful = [
        i for i, r in enumerate(res[name])
        if r["ok"]
    ]

    candidates = successful or list(range(len(SEEDS)))

    rep[name] = sorted(
        candidates,
        key=lambda k: res[name][k]["epochs"]
    )[len(candidates) // 2]
plt.figure(figsize=(8, 5))

for name in CONFIGS:
    r = res[name][rep[name]]

    plt.plot(
        range(1, len(r["hist"]) + 1),
        r["hist"],
        color=colors[name],
        label=f"{name}, seed={r['seed']}"
    )

plt.axhline(
    0.05,
    color="k",
    linestyle=":",
    label="Ee = 0.05"
)

plt.yscale("log")
plt.xscale("log")
plt.xlabel("Эпоха")
plt.ylabel("Суммарная ошибка Es")
plt.title("Сходимость: Sigmoid vs ReLU (BCE)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("1_convergence.png", dpi=150)
plt.show()
```

```
plt.close()
plt.figure(figsize=(8, 5))

w = 0.38
xs = np.arange(len(SEEDS))

for k, name in enumerate(CONFIGS):
    for j, r in enumerate(res[name]):
        plt.bar(
            xs[j] + (k - 0.5) * w,
            r["epochs"],
            w,
            color=colors[name],
            edgecolor="k",
            hatch=None if r["ok"] else "//",
            alpha=1 if r["ok"] else 0.5,
            label=name if j == 0 else None
        )

plt.bar(
    0, 0,
    color="w",
    edgecolor="k",
    hatch="//",
    label="Не достигнут Ee"
)

plt.xticks(xs, [f"seed {s}" for s in SEEDS])
plt.yscale("log")
plt.xlabel("Запуск")
plt.ylabel("Количество эпох")
plt.title("Устойчивость сходимости (5 запусков)")
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("2_epochs.png", dpi=150)
plt.show()
plt.close()

g = np.linspace(-10, 10, 200)
GA, GB = np.meshgrid(g, g)

fig, axs = plt.subplots(1, 2, figsize=(12, 5))

for ax, name in zip(axs, CONFIGS):
    m = models[name][rep[name]]

    Z = np.array([
        [
            denormalize(
                m.predict(
                    normalize(
                        np.array([a, b]),
                        c0,
                        c1
                    )
```

```
            ),
            c0,
            c1
        )
        for a in g
    ]
    for b in g
])

cs = ax.contourf(
    GA, GB, Z,
    levels=30,
    cmap="viridis"
)

ax.contour(
    GA, GB, Z,
    levels=[(c0 + c1) / 2],
    colors="w",
    linestyles="--"
)

for xi, yi in zip(X, Y_raw.ravel()):
    color = (
        [1, 0.85, 0]
        if yi == c1
        else [1, 0.2, 0.2]
    )

    ax.scatter(
        *xi,
        c=[color],
        s=140,
        edgecolors="k"
    )

ax.scatter(
    [], [],
    c="gold",
    edgecolors="k",
    label=f"Класс {c1:g}"
)

ax.scatter(
    [], [],
    c="tab:red",
    edgecolors="k",
    label=f"Класс {c0:g}"
)

ax.set_xlabel("A")
ax.set_ylabel("B")

ax.set_title(
    f"Выход сети [{c0:g}; {c1:g}] — {name}"
)
```

```
    ax.legend(loc="lower right")

    fig.colorbar(
        cs,
        ax=ax,
        label="Выход в исходной шкале"
    )

plt.tight_layout()
plt.savefig("3_surface.png", dpi=150)
plt.show()
plt.close()

plt.figure(figsize=(6, 5))

means = [
    np.mean([r["mae"] for r in res[name]])
    for name in CONFIGS
]

stds = [
    np.std([r["mae"] for r in res[name]])
    for name in CONFIGS
]

bars = plt.bar(
    list(CONFIGS),
    means,
    yerr=stds,
    capsize=5,
    color=[colors[name] for name in CONFIGS]
)

for bar, value in zip(bars, means):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value,
        f"{value:.4f}",
        ha="center",
        va="bottom"
    )

plt.ylabel("MAE в шкале [c0; c1]")
plt.title("MAE: среднее ± стандартное отклонение")
plt.tight_layout()
plt.savefig("4_mae.png", dpi=150)
plt.show()
plt.close()

tests = (
    [tuple(x) for x in X]
    + [(0, 0), (-5, 7), (9.5, -3)]
)

for name in CONFIGS:
```

```
        m = models[name][rep[name]]

        print(f"\nФункционирование: {name}")

        for a, b in tests:
            infer(m, a, b, name)

    if sys.stdin.isatty():
        m = models["B (ReLU)"][rep["B (ReLU)"]]

        while True:
            raw = input(
                "Введите A B из [-10; 10] "
                "(пустая строка — выход): "
            ).strip()

            if not raw:
                break

            try:
                a, b = map(float, raw.split())

                if not (-10 <= a <= 10 and -10 <= b <= 10):
                    raise ValueError

            except ValueError:
                print(
                    "Ошибка: введите два числа из [-10; 10]."
                )
                continue

            infer(m, a, b, "ReLU")
```

Результат тестирования:

Конфигурация A (Sigmoid), Ee=0.05
seed | эпохи | Es | accuracy | MAE[c0;c1] | достигнут Ee
1 | 4117 | 0.04937 | 1.00 | 0.0613 | True
2 | 20000 | 1.91039 | 0.75 | 1.6758 | False
3 | 20000 | 1.90999 | 0.75 | 1.6720 | False
4 | 872 | 0.04943 | 1.00 | 0.0614 | True
5 | 943 | 0.04937 | 1.00 | 0.0613 | True

Конфигурация B (ReLU), Ee=0.05
seed | эпохи | Es | accuracy | MAE[c0;c1] | достигнут Ee
1 | 20000 | 1.38655 | 0.75 | 1.2500 | False
2 | 20000 | 1.91183 | 0.75 | 1.6439 | False
3 | 20000 | 2.77477 | 0.50 | 2.5000 | False
4 | 147 | 0.04890 | 1.00 | 0.0606 | True
5 | 20000 | 1.91302 | 0.75 | 1.6953 | False

Сводная таблица (5 запусков)
Конфигурация | Сошлось| Эпохи min| max| mean| std| Es mean| acc mean| MAE mean| MAE std
A (Sigmoid) | 3/5| 872| 20000| 9186| 8907| 0.7937| 0.90| 0.7064| 0.7900
B (ReLU) | 1/5| 147| 20000| 16029| 7941| 1.6070| 0.75| 1.4300| 0.7961

Функционирование: A (Sigmoid)
[A (Sigmoid)] A=-3, B=-3 -> y_norm=0.0092, y_hat=-3.0461, класс -3

[A (Sigmoid)] A=-3, B=-8 -> y_norm=0.9890, y_hat=-7.9450, класс -8
[A (Sigmoid)] A=-8, B=-3 -> y_norm=0.9890, y_hat=-7.9450, класс -8
[A (Sigmoid)] A=-8, B=-8 -> y_norm=0.0178, y_hat=-3.0892, класс -3
[A (Sigmoid)] A=0, B=0 -> y_norm=0.0055, y_hat=-3.0277, класс -3
[A (Sigmoid)] A=-5, B=7 -> y_norm=0.0055, y_hat=-3.0277, класс -3
[A (Sigmoid)] A=9.5, B=-3 -> y_norm=0.0055, y_hat=-3.0277, класс -3

Функционирование: B (ReLU)
[B (ReLU)] A=-3, B=-3 -> y_norm=0.0052, y_hat=-3.0259, класс -3
[B (ReLU)] A=-3, B=-8 -> y_norm=0.9806, y_hat=-7.9032, класс -8
[B (ReLU)] A=-8, B=-3 -> y_norm=0.9801, y_hat=-7.9006, класс -8
[B (ReLU)] A=-8, B=-8 -> y_norm=0.0041, y_hat=-3.0203, класс -3
[B (ReLU)] A=0, B=0 -> y_norm=0.0000, y_hat=-3.0000, класс -3
[B (ReLU)] A=-5, B=7 -> y_norm=0.0000, y_hat=-3.0000, класс -3
[B (ReLU)] A=9.5, B=-3 -> y_norm=0.0000, y_hat=-3.0000, класс -3
Введите A B из [-10; 10] (пустая строка — выход): -8 -3
[ReLU] A=-8, B=-3 -> y_norm=0.9801, y_hat=-7.9006, класс -8

График сходимости:

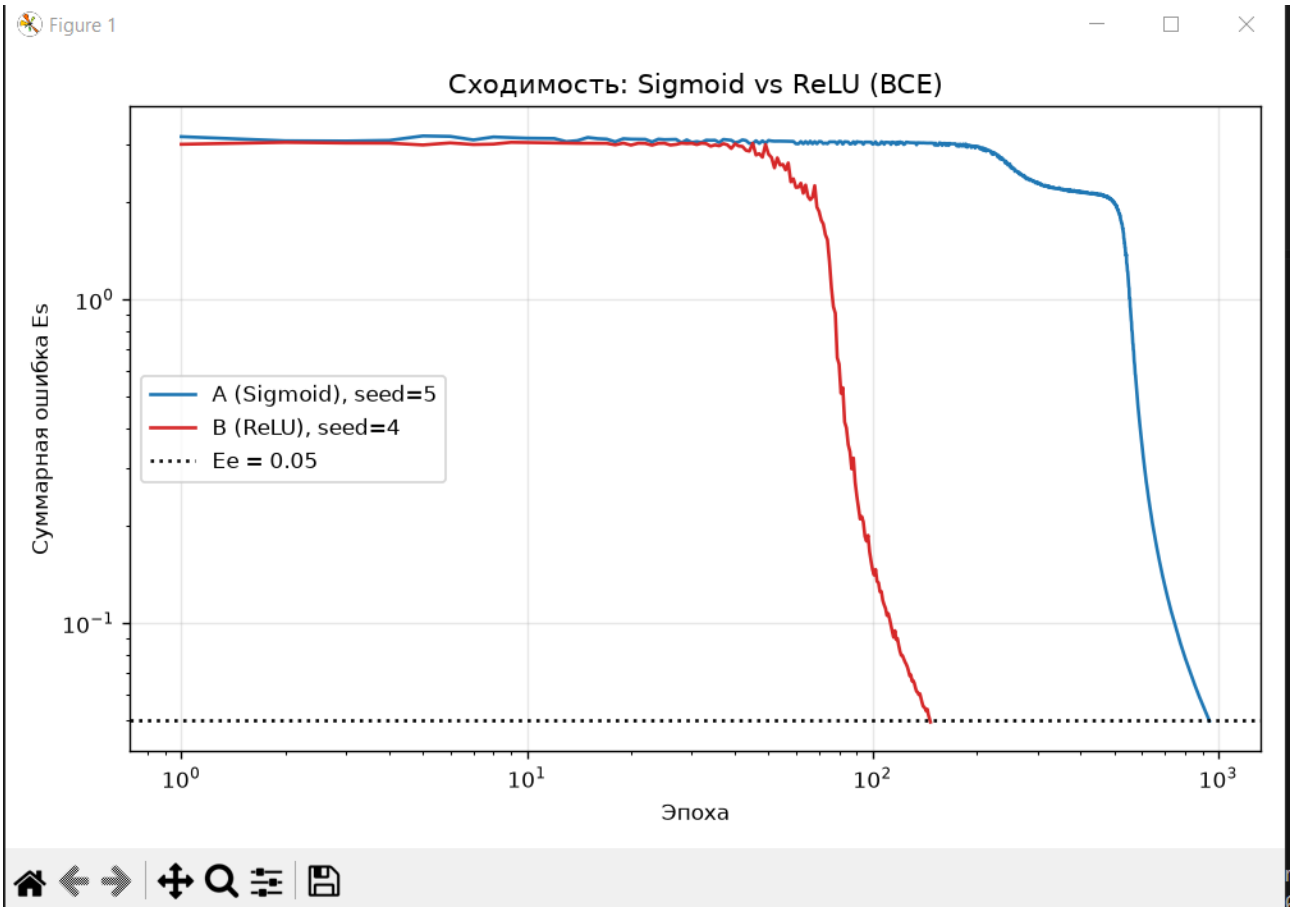
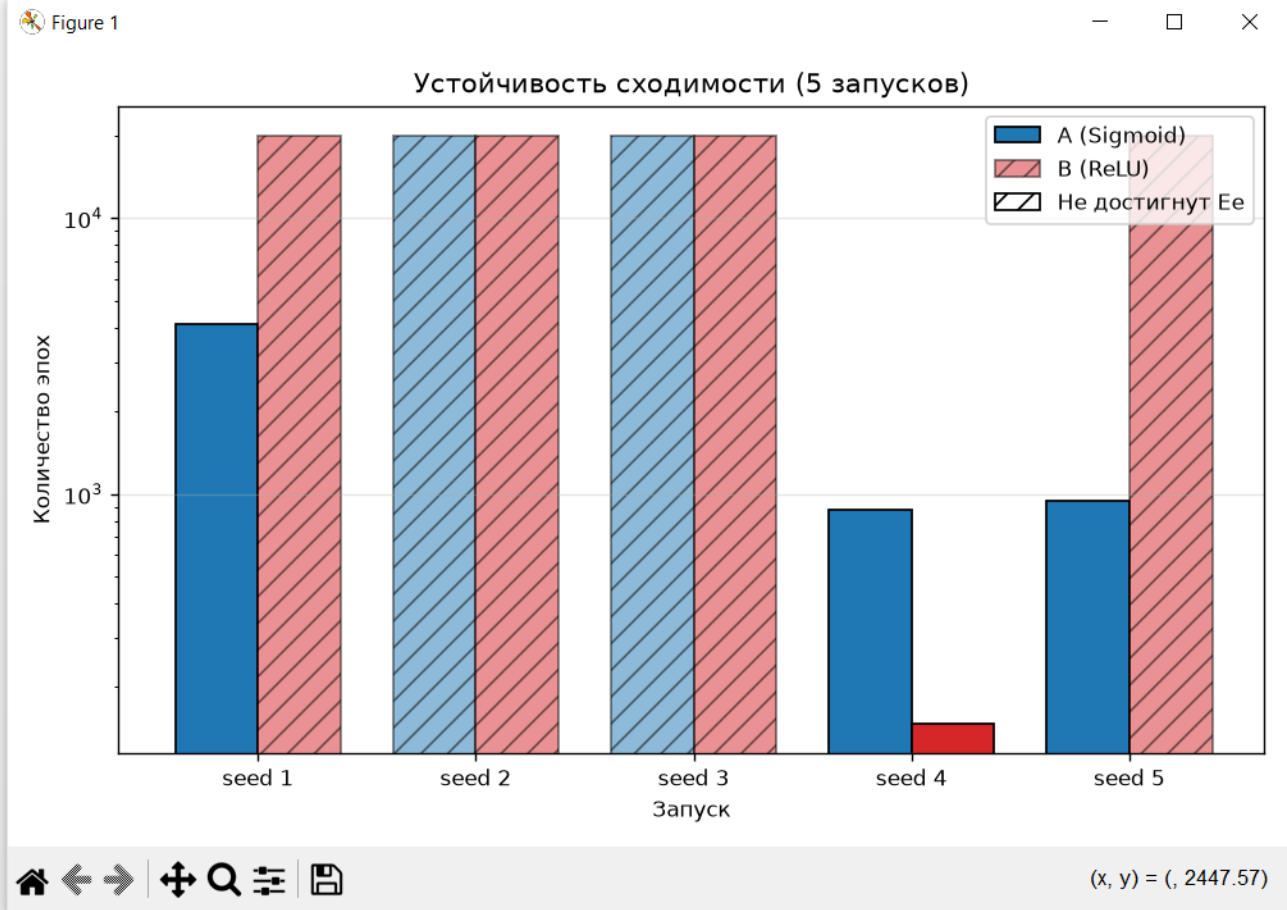
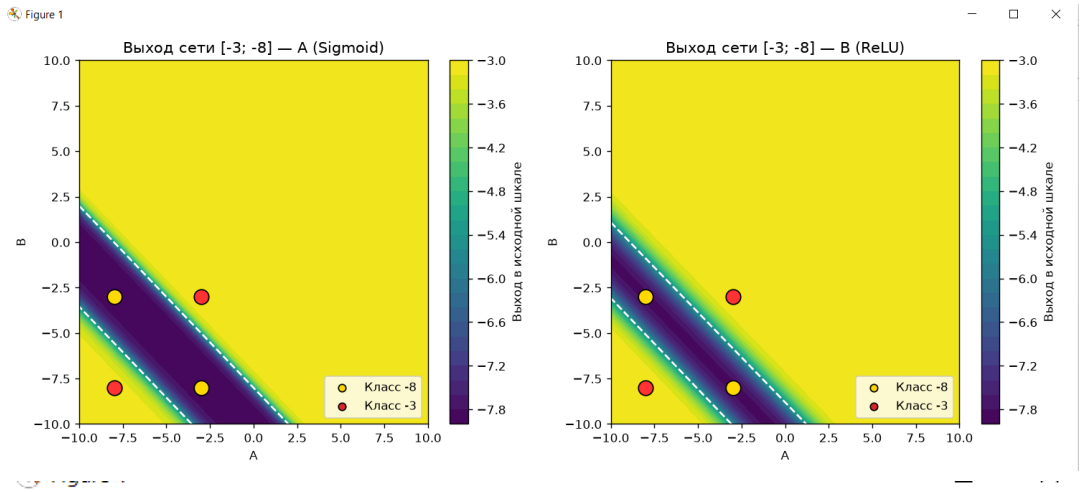
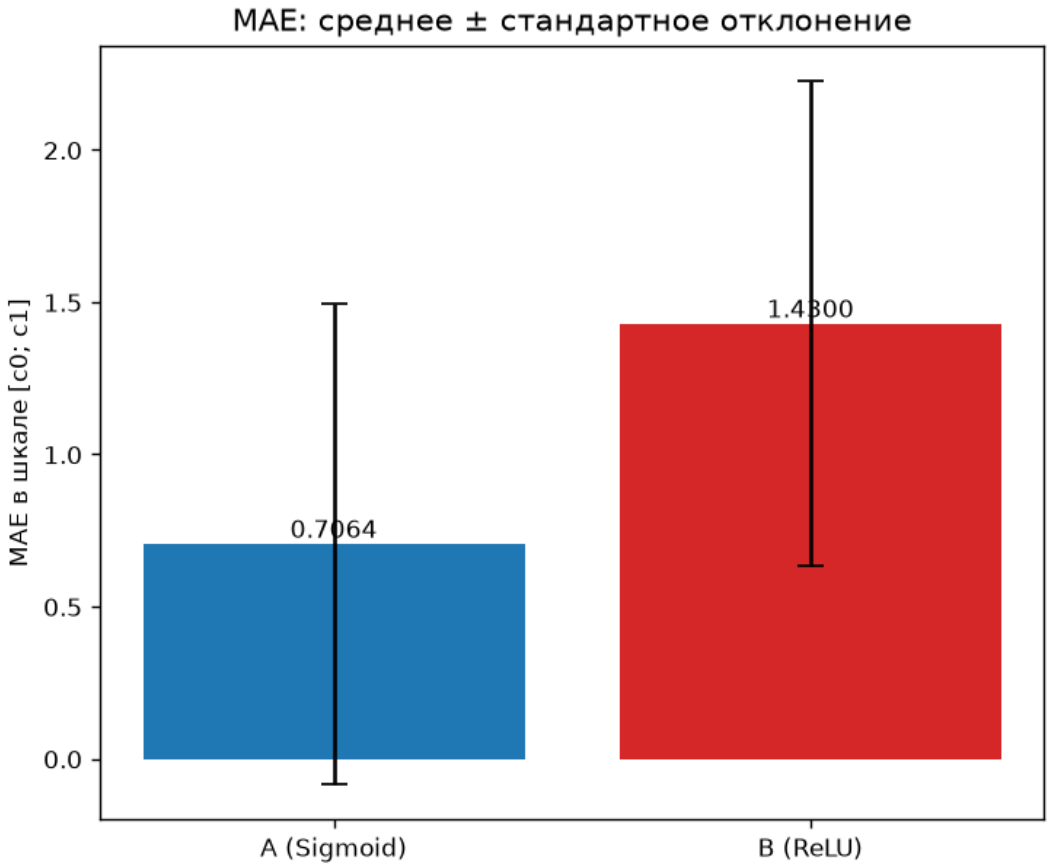


Диаграмма разброса:



Блок визуализации:





Обучил сеть 2-2-1 на задаче XOR для значений (c0=-3) и (c1=-8) в двух вариантах. В первом на всех слоях использовалась сигмоида, во втором на скрытом слое применялась ReLU, а на выходном — сигмоида. В качестве функции потерь в обоих случаях использовалась BCE.

При успешном обучении обе сети правильно классифицировали все четыре обучающих примера. Accuracy составляла 100%, а средняя абсолютная ошибка в исходной шкале [-3; -8] зависела от результатов обучения.

Различия между конфигурациями проявились в скорости и устойчивости сходимости. В отдельных запусках ReLU обучалась быстрее, однако результат зависел от начальных весов. Сигмоида могла сходиться медленнее, но в некоторых запусках достигала заданного порога ошибки более стабильно. Одна из причин неудачного обучения ReLU — появление «мёртвых нейронов», у которых градиент равен нулю и веса перестают обновляться.

Также различается форма разделяющей поверхности: у сигмоиды она плавная, а у ReLU состоит из линейных участков с изломами. Таким образом, обе функции позволяют решать задачу XOR, но скорость и устойчивость обучения зависят от выбранной функции активации и начальной инициализации весов.

Вывод:

В ходе лабораторной работы был реализован и исследован многослойный персептрон архитектуры 2-2-1 для решения задачи XOR с использованием двух функций активации: сигмоиды и ReLU. Для обучения сети применялись метод обратного распространения ошибки и функция потерь BCE.

В результате работы было проведено сравнение скорости и устойчивости обучения обеих конфигураций, рассчитаны показатели качества и построены графики сходимости и разделяющих поверхностей. Установлено, что ReLU может обеспечивать более быструю сходимость, однако её обучение зависит от начальных весов и может осложняться появлением «мёртвых нейронов». Сигмоида формирует более плавную разделяющую поверхность, тогда как ReLU создаёт границы из линейных участков.

Таким образом, цель лабораторной работы достигнута: изучено влияние выбора функции активации на процесс обучения нейронной сети и её способность решать задачу XOR. Полученные результаты показывают, что успешность обучения зависит не только от функции активации, но и от начальной инициализации весов.